robot-control Plugin Tool

robot-control Plugin Tool

1. Course Content
   - Course Overview
   - Learning Objectives
2. Preparation
3. OpenClaw Plugin
4. Plugin Tool Management
5. Source Code Analysis
   - 5.1 Plugin Entry and Tool Registration
   - 5.2 MCP Client Communication

1. Course Content

Course Overview

- The robot-control plugin is the core robot control plugin of OpenClaw. As a bridge between the OpenClaw AI and the robot hardware, it converts the user's natural language instructions into specific robot operation instructions through the MCP protocol.

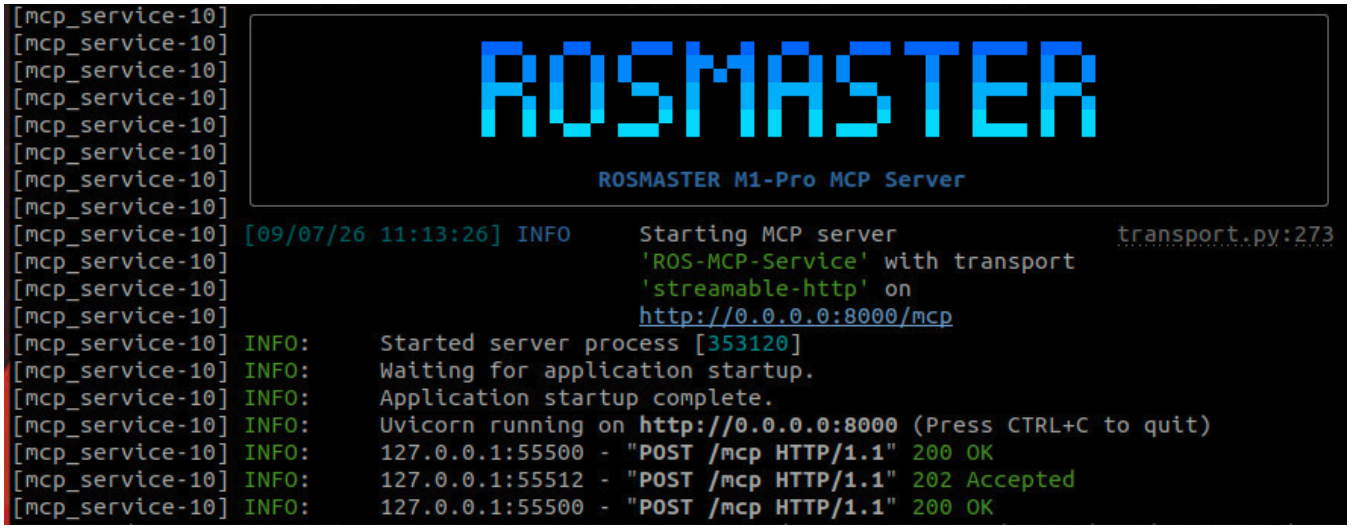
Learning Objectives

1. Understand the role and functional positioning of the robot-control plugin in the OpenClaw framework
2. Master how to install, view, and perform basic management operations on plugins
3. Be able to manage and call robot control tools through the OpenClaw WebChat interface

> **Important**
>
> The demonstration in this lesson uses a depth camera (hp60c) model as an example; the MCP and CLI tools may differ slightly between hardware versions.

2. Preparation

```
ros2 launch yahboomcar_bringup mcp_service.launch.py use_camera:=true
```



3. OpenClaw Plugin

- Plugin feature introduction: The OpenClaw plugin system is the feature extension mechanism of the OpenClaw platform. By installing different plugins, various capabilities can be added to OpenClaw. The core plugins related to robot control currently include:
- **Robot-Control Plugin**: The core robot control plugin, providing all robot operation functions such as chassis movement, robotic arm control, visual recognition, and navigation/localization

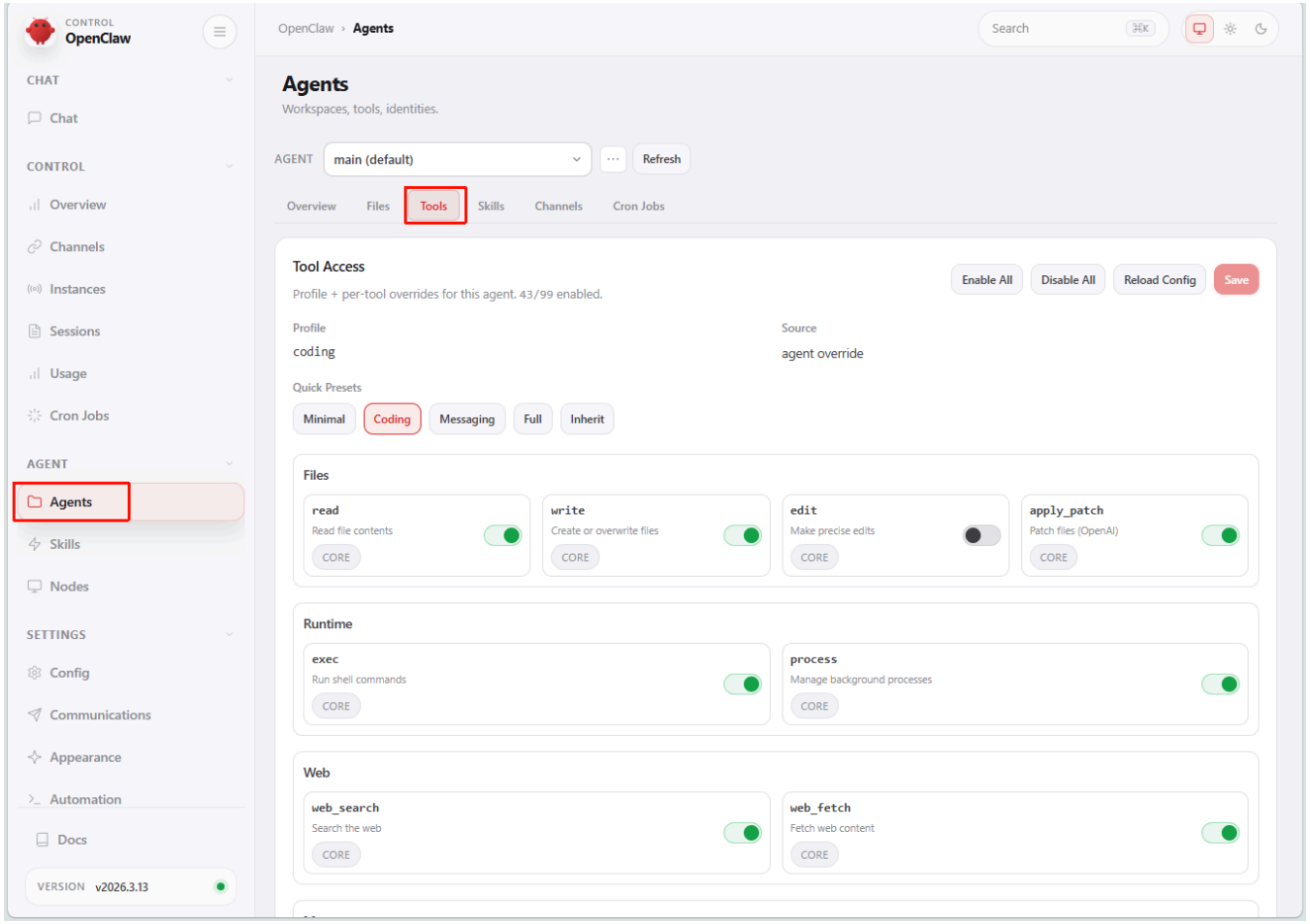
```
openclaw plugins list
```

- Robot-Control in the plugin list is the plugin used to control the robot with OpenClaw

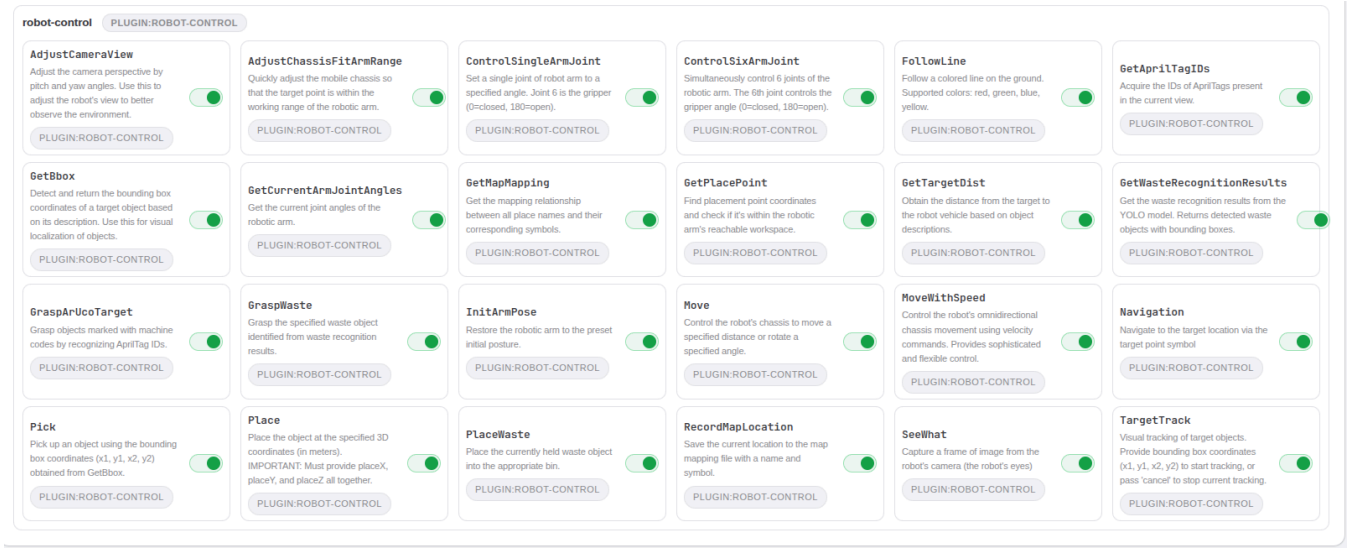


4. Plugin Tool Management

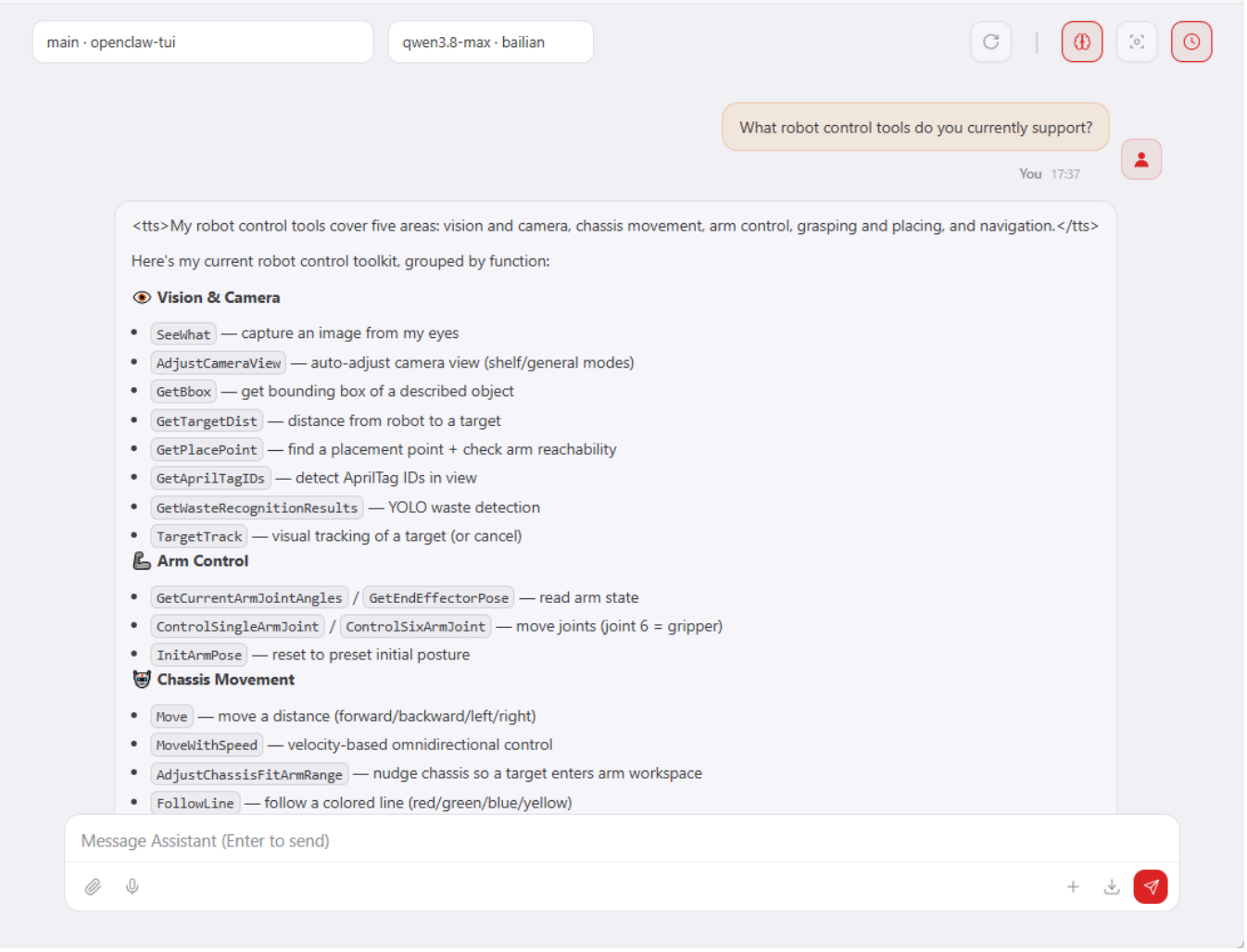
- Log in to the OpenClaw WebChat interface, and select Agent — Tools in sequence



- Scroll down to find the robot-control plugin tool list, where you can view and manage all plugin tools
    - All plugin features of robot-control are enabled by default at the factory (the features correspond to the MCP and CLI interfaces). If you need to manually disable a tool, click to turn off the corresponding tool, click Save to save, and restart the OpenClaw gateway



- You can directly ask OpenClaw to view the currently available robot control tools



5. Source Code Analysis

5.1 Plugin Entry and Tool Registration

Source code directory:

```
~/.openclaw/extensions/robot-control/index.ts
```

The plugin source code is located in `index.ts` under the plugin directory. It uses the `registerRobotControlTools` function as the entry point and registers all robot control tools using the OpenClaw Plugin SDK. Take the `Move` (chassis movement) tool as an example:

```
export default function registerRobotControlTools(api: OpenClawPluginApi) {
  // Load the SeeWhat channel configuration at startup
  void loadSeeWhatConfig(api.config as Record<string, unknown> | undefined);

  api.registerTool(
    {
      name: "Move",
      label: "Move",
      description: "Control the robot's chassis to move a specified distance.",
      parameters: MoveSchema,
```

```
      async execute(_toolCallId, params) {
        try {
          const args = buildArgs({ "distance": params.distance, "direction":
params.direction });
          const result = await callRobotControl("Move", args);
          return json(result);
        } catch (err) {
          return json({ error: err instanceof Error ? err.message : String(err) });
        }
      },
    },
    { name: "Move" },
  );
  // ... other tool registrations
}
```

Code Description:

1. `api.registerTool()`: The tool registration method provided by the SDK, passing in the tool definition (name, description, parameters Schema, execute function) and metadata
2. **Parameter Schema**: Uses `Type.Object()` of `@sinclair/typebox` to define the parameter structure and constraints
3. **Execute function**: `buildArgs` converts the parameters into CLI format and calls the robot control interface through `callRobotControl`

5.2 MCP Client Communication

The plugin internally encapsulates the `McpClient` class, which communicates with the MCP server via JSON-RPC over the HTTP protocol:

`McpClient`: Maintains the MCP session state, automatically initializes on the first call, and sends tool call requests through the JSON-RPC protocol

```
const MCP_ENDPOINT = "http://localhost:8000/mcp";

class McpClient {
  private sessionId: string | null = null;
  private initialized = false;

  async ensureInitialized(): Promise<void> {
    if (this.initialized) return;
    await this.sendRequest("initialize", {
      protocolVersion: "2025-03-26",
      capabilities: {},
      clientInfo: { name: "openclaw-plugin", version: "1.0.0" },
    });
    await this.sendNotification("notifications/initialized");
    this.initialized = true;
  }

  async callTool(name: string, args: Record<string, unknown>) {
```

```
    await this.ensureInitialized();
    return await this.sendRequest("tools/call", { name, arguments: args });
  }
}
```

Code Description:

- `MCP_ENDPOINT`: MCP server address, fixed at `http://localhost:8000/mcp`; the plugin communicates with the MCP server via JSON-RPC through this address
- `McpClient`: MCP client class that maintains the session state and automatically initializes the connection on the first call
- `sessionId`: MCP session ID, assigned by the server during initialization, used to identify the current communication session
- `initialized`: Initialization flag, ensuring the MCP handshake protocol is executed only once
- `ensureInitialized()`: A lazy initialization method that automatically performs the MCP initialization handshake on the first call and returns directly on subsequent calls
- `sendRequest("initialize", {...})`: Sends the MCP initialization request, carrying the protocol version `protocolVersion`, client capabilities `capabilities`, and client information `clientInfo`
- `sendNotification("notifications/initialized")`: Sends the initialization completion notification, informing the server that the client is ready
- `callTool(name, args)`: Calls an MCP tool; internally it first ensures initialization is complete, then sends the tool call request through the `tools/call` method, where `name` is the tool name and `args` are the tool parameters